

Лабораторная работа № 1

Численное интегрирование

Выполнили студенты 2 курса группы ИВТ1.1:

Зубов М.Г.

Шульга Е.Е.

Постановка задачи.

Вычислить определенный интеграл, используя различные численные методы и алгоритмы их реализации. Провести сравнительный анализ полученных результатов. Сделать вывод.

1. Программный модуль Меню

Его назначение:

1. Позволяет пользователю выбрать тип решаемой задачи
2. Позволяет пользователю выбрать численный метод для решения поставленной задачи.
3. Позволяет пользователю выбрать алгоритм решаемой задачи.

Код программы:

```
from tkinter import *
from tkinter import messagebox
from math import *
from multipleIntegration import *
from variablestep_algorithm import *
from constantstep_algorithm import *
from integration import *

def calculate():
    result = ''
    match task_type_option.get():
        case 1:
            a_raw = a_input.get()
            b_raw = b_input.get()
            n_raw = n_input.get()
            formula_raw = formula_input.get()

            a = float(eval(a_raw))
            b = float(eval(b_raw))
            n = int(eval(n_raw))
            tkGetFormula(formula_raw)

            match methods_type_option.get():
                case 1:

                    match algorithm_type_option.get():
                        case 1:
                            result = f'Результат: {left_rectangle(a, b, n)}'
                        case 2:
                            E_raw = E_input.get()
                            E = float(E_raw)
```

```

        result = f'Результат: {var_left_rectangle(a, b, n, E)}'
    case 3:
        E_raw = E_input.get()
        E = float(E_raw)
        result = f'Результат: {var_second_left_rectangle(a, b, n, E)}'
case 2:

    match algorithm_type_option.get():
        case 1:
            result = f'Результат: {right_rectangle(a, b, n)}'
        case 3:

            match algorithm_type_option.get():
                case 1:
                    result = f'Результат: {trapezoid(a, b, n)}'
            case 4:

                match algorithm_type_option.get():
                    case 1:
                        result = f'Результат: {parabola(a, b, n)}'
case 2:
    a_raw = a_input.get()
    b_raw = b_input.get()
    n_raw = n_input.get()
    c_raw = c_input.get()
    d_raw = d_input.get()
    ny_raw = ny_input.get()
    formula_raw = formula_input.get()

    a = float(eval(a_raw))
    b = float(eval(b_raw))
    n = int(eval(n_raw))
    c = float(eval(c_raw))
    d = float(eval(d_raw))
    ny = int(eval(ny_raw))
    tkGetMultipleFormula(formula_raw)

    result = f'Результат: {re_integration(a, b, c, d, n, ny)}'
return result

def show_result():
    result = calculate()
    reslbl.configure(text=result)
    reslbl.grid(column=1, row=5)

```

```

def var_algorithm_frame_render():
    algorithm_type_option.set(0)
    for widget in algorithm_frame.winfo_children():
        widget.destroy()

    algorithmlbl = Label(algorithm_frame, text="Алгоритм:", padx=5, pady=5)
    double_algorithm_btn = Radiobutton(algorithm_frame, text="Двойной пересчет",
variable=algorithm_type_option, value=2, command=input_frame_render)
    second_double_algorithm_btn = Radiobutton(algorithm_frame, text="Двойной
пересчет с отступами", variable=algorithm_type_option, value=3,
command=input_frame_render)
    algorithmlbl.grid(column=0, row=0)
    double_algorithm_btn.grid(column=0, row=1)
    second_double_algorithm_btn.grid(column=0, row=2)

def algorithm_frame_render():
    for widget in algorithm_frame.winfo_children():
        widget.destroy()
    match methods_type_option.get():
        case 1:
            algorithmlbl = Label(algorithm_frame, text="Алгоритм:", padx=5, pady=5)
            constantstep_btn = Radiobutton(algorithm_frame, text="Постоянный шаг",
variable=algorithm_type_option, value=1, command= input_frame_render)
            variablestep_btn = Radiobutton(algorithm_frame, text="Переменный шаг",
variable=algorithm_type_option, value=2, command= var_algorithm_frame_render)
            algorithmlbl.grid(column=0, row=0)
            constantstep_btn.grid(column=0, row=1)
            variablestep_btn.grid(column=0, row=2)
        case 2:
            algorithmlbl = Label(algorithm_frame, text="Алгоритм:", padx=5, pady=5)
            constantstep_btn = Radiobutton(algorithm_frame, text="Постоянный шаг",
variable=algorithm_type_option, value=1, command=input_frame_render)
            algorithmlbl.grid(column=0, row=0)
            constantstep_btn.grid(column=0, row=1)
        case 3:
            algorithmlbl = Label(algorithm_frame, text="Алгоритм:", padx=5, pady=5)
            constantstep_btn = Radiobutton(algorithm_frame, text="Постоянный шаг",
variable=algorithm_type_option, value=1, command= input_frame_render)
            algorithmlbl.grid(column=0, row=0)
            constantstep_btn.grid(column=0, row=1)
        case 4:
            algorithmlbl = Label(algorithm_frame, text="Алгоритм:", padx=5, pady=5)
            constantstep_btn = Radiobutton(algorithm_frame, text="Постоянный шаг",
variable=algorithm_type_option, value=1, command= input_frame_render)

```

```

algorithmlbl.grid(column=0, row=0)
constantstep_btn.grid(column=0, row=1)

def methods_frame_render():
    methods_frame.grid_forget()
    methods_frame.grid(column=1, row=1)
    methodslbl = Label(methods_frame, text="Метод:", padx=5, pady=5)
    left_rectangle_btn = Radiobutton(methods_frame, text="Левых прямоугольников",
variable=methods_type_option, value=1, command=algorithm_frame_render)
    right_rectangle_btn = Radiobutton(methods_frame, text="Правых
прямоугольников", variable=methods_type_option, value=2,
command=algorithm_frame_render)
    trapezoid_btn = Radiobutton(methods_frame, text="Трапеций",
variable=methods_type_option, value=3, command=algorithm_frame_render)
    parabola_btn = Radiobutton(methods_frame, text="Парабол",
variable=methods_type_option, value=4, command=algorithm_frame_render)
    methodslbl.grid(column=0, row=0)
    left_rectangle_btn.grid(column=0, row=1)
    right_rectangle_btn.grid(column=0, row=2)
    trapezoid_btn.grid(column=0, row=3)
    parabola_btn.grid(column=0, row=4)

def input_frame_render():
    for widget in input_frame.winfo_children():
        widget.grid_forget()

    match task_type_option.get():
        case 1:
            match algorithm_type_option.get():
                case 1:
                    blanklbl = Label(input_frame, text=" ", padx=5, pady=5)
                    albl = Label(input_frame, text="Введите a: ", padx=5, pady=5)
                    blbl = Label(input_frame, text="Введите b: ", padx=5, pady=5)
                    nlbl = Label(input_frame, text="Введите n: ", padx=5, pady=5)
                    formulalbl = Label(input_frame, text="Введите выражение: ", padx=5,
pady=5)
                    blanklbl.grid(column=0, row=0)
                    albl.grid(column=0, row=1)
                    blbl.grid(column=0, row=2)
                    nlbl.grid(column=0, row=3)
                    formulalbl.grid(column=0, row=4)
                    a_input.grid(column=1, row=1)
                    b_input.grid(column=1, row=2)
                    n_input.grid(column=1, row=3)

```

```
formula_input.grid(column=1, row=4)
```

case 2:

```
blanklbl = Label(input_frame, text=" ", padx=5, pady=5)
abl = Label(input_frame, text="Введите a: ", padx=5, pady=5)
blbl = Label(input_frame, text="Введите b: ", padx=5, pady=5)
nlbl = Label(input_frame, text="Введите n: ", padx=5, pady=5)
Elbl = Label(input_frame, text="Введите E: ", padx=5, pady=5)
formulabl = Label(input_frame, text="Введите выражение: ", padx=5,
pady=5)
```

```
blanklbl.grid(column=0, row=0)
abl.grid(column=0, row=1)
blbl.grid(column=0, row=2)
nlbl.grid(column=0, row=3)
Elbl.grid(column=0, row=4)
formulabl.grid(column=0, row=5)
a_input.grid(column=1, row=1)
b_input.grid(column=1, row=2)
n_input.grid(column=1, row=3)
E_input.grid(column=1, row=4)
formula_input.grid(column=1, row=5)
```

case 3:

```
blanklbl = Label(input_frame, text=" ", padx=5, pady=5)
abl = Label(input_frame, text="Введите a: ", padx=5, pady=5)
blbl = Label(input_frame, text="Введите b: ", padx=5, pady=5)
nlbl = Label(input_frame, text="Введите n: ", padx=5, pady=5)
Elbl = Label(input_frame, text="Введите E: ", padx=5, pady=5)
formulabl = Label(input_frame, text="Введите выражение: ", padx=5,
pady=5)
```

```
blanklbl.grid(column=0, row=0)
abl.grid(column=0, row=1)
blbl.grid(column=0, row=2)
nlbl.grid(column=0, row=3)
Elbl.grid(column=0, row=4)
formulabl.grid(column=0, row=5)
a_input.grid(column=1, row=1)
b_input.grid(column=1, row=2)
n_input.grid(column=1, row=3)
E_input.grid(column=1, row=4)
formula_input.grid(column=1, row=5)
```

case 2:

```
blanklbl = Label(input_frame, text=" ", padx=5, pady=5)
abl = Label(input_frame, text="Введите a: ", padx=5, pady=5)
blbl = Label(input_frame, text="Введите b: ", padx=5, pady=5)
clbl = Label(input_frame, text="Введите c: ", padx=5, pady=5)
```

```

dlbl = Label(input_frame, text="Введите d: ", padx=5, pady=5)
nlbl = Label(input_frame, text="Введите nX: ", padx=5, pady=5)
nylbl = Label(input_frame, text="Введите nY: ", padx=5, pady=5)
formulabl = Label(input_frame, text="Введите выражение: ", padx=5, pady=5)
blanklbl.grid(column=0, row=0)
albl.grid(column=0, row=1)
blbl.grid(column=0, row=2)
nlbl.grid(column=0, row=3)
formulabl.grid(column=0, row=4)
clbl.grid(column=2, row=1)
dlbl.grid(column=2, row=2)
nylbl.grid(column=2, row=3)
formulabl.grid(column=0, row=4)
a_input.grid(column=1, row=1)
b_input.grid(column=1, row=2)
n_input.grid(column=1, row=3)
c_input.grid(column=3, row=1)
d_input.grid(column=3, row=2)
ny_input.grid(column=3, row=3)
formula_input.grid(column=1, row=4)

```

```

def render_control():
    match task_type_option.get():
        case 1:
            for widget in input_frame.winfo_children():
                widget.grid_forget()
            methods_frame.grid_forget()
            algorithm_frame.grid_forget()
            input_frame.grid(column=0, row=1)
            methods_frame.grid(column=1, row=1)
            algorithm_frame.grid(column=2, row=1)
            methods_frame_render()
        case 2:
            for widget in input_frame.winfo_children():
                widget.grid_forget()
            methods_frame.grid_forget()
            algorithm_frame.grid_forget()
            input_frame.grid(column=0, row=1)
            input_frame_render()

```

```

window = Tk()
window.title("Интегралы")
window.geometry('820x300')

```

```

task_type_option = IntVar()
methods_type_option = IntVar()
algorithm_type_option = IntVar()

Radiobutton(window, text="Численное интегрирование", variable=task_type_option,
value=1, command=render_control).grid(column=0, row=0)
Radiobutton(window, text="Кратные интегралы ", variable=task_type_option, value=2,
command=render_control).grid(column=1, row=0)
Button(window, text="Вычислить", command=show_result, padx=5,
pady=5).grid(row=5, column=0)

input_frame = Frame(window)
methods_frame = Frame(window)
algorithm_frame = Frame(window)

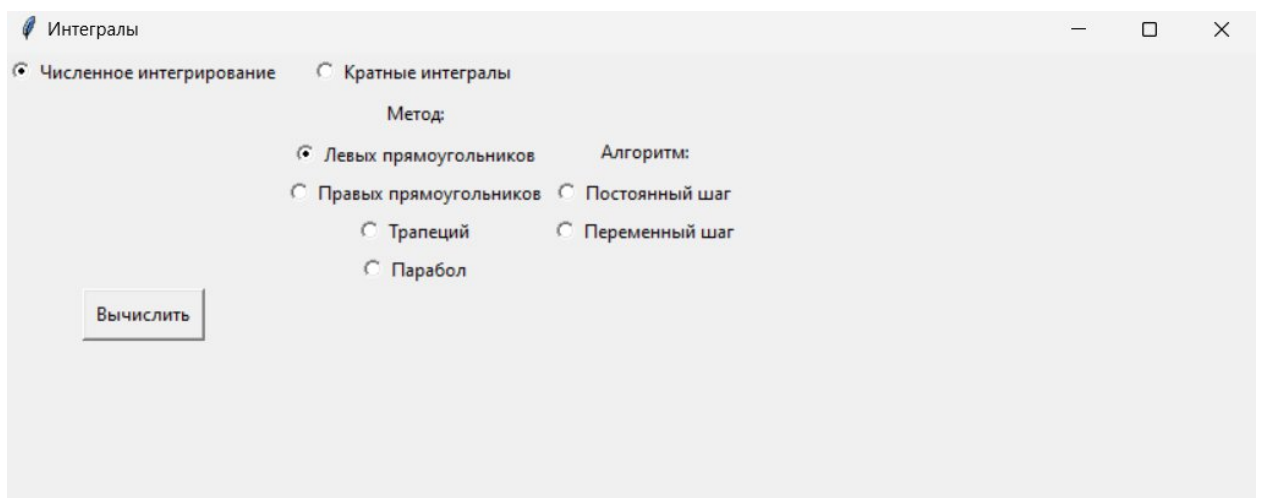
formula_input = Entry(input_frame, width=20)
a_input = Entry(input_frame, width=5)
b_input = Entry(input_frame, width=5)
n_input = Entry(input_frame, width=5)
E_input = Entry(input_frame, width=5)
c_input = Entry(input_frame, width=5)
d_input = Entry(input_frame, width=5)
ny_input = Entry(input_frame, width=5)

reslbl = Label(window, text=' ', padx=5, pady=5)

window.mainloop()

```

Пример работы модуля:



Итог проделанной работы:

Разрабатывая данный модуль мы смогли освоить основы обращения с инструментами библиотеки tkinter для языка программирования Python. В результате работы удалось выполнить поставленную задачу, по созданию Меню со всеми необходимыми возможностями для управления работой ПО(выбор типа решаемой задачи, выбор метода и алгоритма для решения, а также полями ввода значений).

2.Методы и алгоритмы реализованные в ПО.

Метод прямоугольников левых частей

Данный метод включает в себя алгоритмы для решения интегралов с постоянным и переменным шагом.

Алгоритм для решения интегралов с постоянным шагом.

Таблица переменных:

Имя переменной	Смысл переменной	Тип переменной
a	Нижний предел интегрирования	float
b	Верхний предел интегрирования	float
n	Число разбиений	integer
s	Сумма значений подынтегральной функции.	float
h	Шаг интегрирования	float
x	Точка на оси OX	float

Код:

```
def left_rectangle(a, b, n):  
    s = 0  
    h = (b - a) / n  
    x = a  
  
    while x <= b - h:  
        s += f(x)  
        x += h  
  
    result = h * s  
    return result
```

Пример работы модуля:

Интегралы

☒ Численное интегрирование ☐ Кратные интегралы

Метод:

☒ Левых прямоугольников ☐ Правых прямоугольников ☐ Трапеций ☐ Парабол

Алгоритм:

☒ Постоянный шаг ☐ Переменный шаг

Введите a: 1

Введите b: 3

Введите n: 10

Введите выражение: x^2

Вычислить

Результат: 6.312

Алгоритмы для решения интегралов с переменным шагом.

1. Двойной пересчет

Таблица переменных:

Имя переменной	Смысл переменной	Тип переменной
a	Нижний предел интегрирования	float
b	Верхний предел интегрирования	float
n	Число разбиений	integer
s2	сумма значений подынтегральной функции.	float
h	Шаг интегрирования	float
x	Точка на оси OX	float
r	разность ($I_{2n} - I_n$)	float
I_n	значение интеграла I_n при числе разбиений n	float
I_{2n}	значение интеграла I_{2n} при числе разбиений 2n	float
E	точность ϵ , с которой ведем счет	float

Код:

```
def var_left_rectangle(a, b, n, E):  
    h = (b - a) / n  
    r = 1  
     $I_n = 0$   
     $I_{2n} = 0$   
    while r > E:  
        s2 = 0  
        x = a  
        while x <= b - h:  
            s2 += f(x)  
            x += h  
         $I_{2n} = h * s2$   
        r = abs( $I_{2n} - I_n$ )  
         $I_n = I_{2n}$   
        h = h/2
```

```
result = I2n
return result
```

Пример работы модуля:

Интегралы

☒ Численное интегрирование ☐ Кратные интегралы

Введите a: 1
Введите b: 3
Введите n: 10
Введите E: 0.01

Метод:
☒ Левых прямоугольников ☐ Правых прямоугольников
☐ Трапеций ☐ Парабол

Алгоритм:
☒ Двойной пересчет ☐ Двойной пересчет с отступами

Введите выражение: x**2

Вычислить

Результат: 8.660417480468205

2. Двойной пересчет с отступом

Таблица переменных:

Имя переменной	Смысл переменной	Тип переменной
a	Нижний предел интегрирования	float
b	Верхний предел интегрирования	float
n	Число разбиений	integer
s1	сумма для интегрирования с шагом h	
s2	сумма для интегрирования с шагом h/2	float
h _v	Шаг вычисления	float
h _s	Шаг смещения	float
h _d	Шаг движения	float
x	Точка на оси OX	float
r	разность (I ₂ – I ₁)	float
I ₁	интеграл, вычисленный с шагом h	float
I ₂	интеграл, вычисленный с шагом h/2	float
E	точность ε, с которой ведем счет	float

Код:

```
def var_second_left_rectangle(a, b, n, E):
    hv = (b - a) / n
    hs = 0
    hd = hv
    I2 = 0
    s1 = 0
    x0 = a
    x = x0

    while x <= b - hv:
        s1 += f(x)
```

```
x += hd

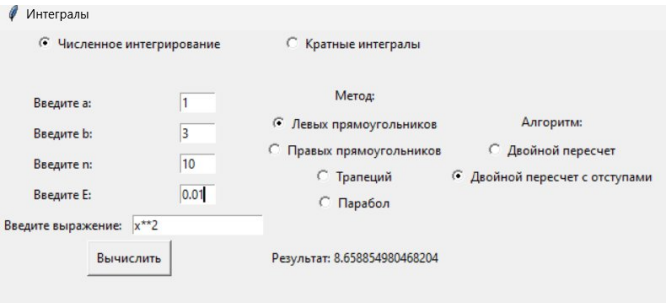
l1 = s1 * hd
r = abs(l2-l1)

while r > E:

    s2 = 0
    x = x0 + hs
    hd = hv
    hv /= 2
    while x <= b - hv:
        s2 += f(x)
        x += hd

    l2 = s2 * hd
    r = abs(l2 - l1)
    l1 = l2
    hs = hd / 2
result = l2
return result
```

Пример работы модуля:



Метод прямоугольников правых частей

Данный метод включает в себя алгоритм для решения интегралов с постоянным шагом.

Таблица переменных:

Имя переменной	Смысл переменной	Тип переменной
a	Нижний предел интегрирования	float
b	Верхний предел интегрирования	float
n	Число разбиений	integer
s	Сумма значений подынтегральной функции.	float
h	Шаг интегрирования	float
x	Точка на оси OX	float

Код:

```
def right_rectangle(a, b, n):
    s = 0
    h = (b - a) / n
    x = a + h

    while x <= b:
        s += f(x)
        x += h

    result = h * s
    return result
```

Пример работы модуля:

Метод трапеций

Данный метод включает в себя алгоритм для решения интегралов с постоянным шагом.

Таблица переменных:

Имя переменной	Смысл переменной	Тип переменной
a	Нижний предел интегрирования	float
b	Верхний предел интегрирования	float
n	Число разбиений	integer
s	Сумма значений подынтегральной функции.	float
h	Шаг интегрирования	float
x	Точка на оси OX	float

Код:

```
def parabola(a, b, n):
    s1 = 0
    s2 = 0
    h = (b - a) / n
    x = a
    s = f(x)
    x = b
    s += f(x)
```

```

x = a + h
while x <= b - h:
    s1 += f(x)
    x += 2 * h
i1 = 4 * s1
x = a + 2 * h
while x <= b - 2 * h:
    s2 += f(x)
    x += 2 * h
i2 = 2 * s2
result = h * (i1 + i2 + s) / 3
return result

```

Пример работы модуля:

Интегралы

☒ Численное интегрирование ☐ Кратные интегралы

Метод:

☐ Левых прямоугольников ☒ Правых прямоугольников ☐ Трапеций ☒ Парабол

Алгоритм:

☒ Постоянный шаг

Введите a:

Введите b:

Введите n:

Введите выражение:

Результат: 8.666666666666666

3.Таблицы результатов вычислений при различном числе разбиений n

Методы для вычисления интегралов с постоянным шагом на примере интеграла

$$\int_3^1 x^2$$

Метод/Число разбиений n	n = 10	n =100	n = 1000	n = 10000
Прямоугольников левых частей	6,312	8,4091920000000008	8,65866799999971	8,665866679999006
Прямоугольников правых частей	7,68	8,5668000000000008	8,674667999999707	8,667466679999006
Трапеций	7,1120000000000001	8,4891900000000008	8,666667999999708	8,666666679999006
Парабол	8,666666666666666	8,313034666666676	8,630730634666675	8,666666666666518

Итог:

Из результатов вычислений можно заметить, что при низком числе разбиений наиболее точным оказался метод парабол, а наименее точным метод левосторонних прямоугольников. Так же видно, что при увеличении числа разбиений, результаты вычислений всех методов становятся более точными и схожими друг с другом.

Алгоритмы метода левосторонних прямоугольников для вычисления интегралов с переменным шагом на примере интеграла $\int_3^1 x^2$ при точности 0.01

Алгоритм/Число разбиений n	n = 10	n = 100	n = 1000	n = 10000
Двойной пересчет	8,660417480468205	8,656668749999795	8,662666999998983	8,665366730003878
Двойной пересчет с отступами	8,658854980468204	8,660417187500276	8,65866799999971	8,665866679999006

Итог:

Из результатов видно, что при вычислении интеграла с помощью алгоритмов с переменным шагом точность вычислений значительно возросла. У обоих алгоритмов точность вычислений находится на примерно одинаковом уровне, однако Двойной пересчет с отступами производит вычисления быстрее, т.к. избегает не нужных вычислений.

4. Вывод

Выполняя данную лабораторную работу, мы ознакомились и реализовали на практике различные методы и алгоритмы для вычисления численных интегралов. Разработали собственное ПО для вычислений. Создали интерфейс для программы, в процессе ознакомившись с новой для нас библиотекой языка Python – tkinter. Собрали данные вычислений, произвели их анализ и сравнили точность и эффективность различных алгоритмов.